Distribuita BD

Grile

→ partea 1_bd

1. SELECT * FROM Sailors WHERE rating > ALL(7,3,10),

! rating trebuie să fie mai mare decât toate valorile din listă

2. Reprezentarea unui set de relații în diagramele ER este un diamant (romb)

3. Când traducem un model ER într-un model relațional, constrângerea care ar trebui adăugată cheii externe ce leagă o entitate slabă de entitatea proprietar ar trebui să fie: ON DELETE CASCADE

4. Modelul Conceptual - folosit pentru a analiza și înțelege datele aplicației

{ Modelul fizic - cum vor fi salvate datele pe disk
{ Modelul de date - înțelegerea bazei (cuprinde toate modelele, conceptual, logic, fizic)

5. SELECT -II- WHERE rating > ANY(7,3,10)

! rating trebuie să fie strict mai mare decât cel puțin unul din listă

6. Produs cartezian - operație care combină fiecare rând din primul tabel cu fiecare rând din al doilea tabel

Gradul - nr. total de coloane dintr-un tabel

Cardinalitatea - nr. total de rânduri dintr-un tabel

În ceea ce privește relația dintre gradul / cardinalitatea relației rezultate și tabelele de intrare ale unui produs cartezian: gradul relației rezultate va fi mai mare, iar cardinalitatea va fi mai mare comparativ cu fiecare tabel de intrare.

Ex	tabel A	tabel B
grad.	3 coloane	2 coloane
card.	10 rânduri	5 rânduri

prod. cartezian ⟹ rezultat: grad. 5, card. 50

grad → +

card → x

7. O bază de date relațională se află într-o stare consistentă dacă toate constrângerile de integritate definite sunt îndeplinite.

8. Un Set de Relații în modelul ER:
- NU conține entități
- Conține EXCLUSIV relații
- același set de entități poate participa la seturi de relații diferite
- același set de entități poate participa cu roluri diferite în seturi de relații diferite
- reprezintă o colecție de relații similare

9. Reprezentarea unui set de entități în diagramele ER este un dreptunghi

10. Axiome Armstrong ← Reflexivitate, Tranzitivitate, Augmentare

11. În ceea ce privește relația dintre gradul/cardinalitatea relației rezultate și tabelul de intrare în cazul operatorului relațional de proiecție:

gradul relației rezultate va fi mai mic sau egal, iar cardinalitatea va fi mai mică sau egală comparativ cu tabelul de intrare.

La proiecție:
→ la grad, se aleg o parte din coloanele tabelului inițial sau toate => nr. de coloane va fi mereu mai mic sau egal
→ la card., mereu nr. final de rânduri va fi mai mic sau egal

12. Tipul de participare a unui set de entități slabe în relația cu proprietarul în modelul ER e participarea totală.

13. O linie subțire folosită pt. a lega o entitate de o relație denotă o constrângere de participare parțială.

Linia groasă → participare totală

Scanned with CamScanner

14. Modul în care datele sunt stocate de un RDBMS pe un suport extern e exprimat de modelul fizic.

15. Dezavantaje fișier gestionat de sistemul de fișiere al sist. de operare (atunci când e folosit ca mediu de stocare pt. o bază de date):
- protecție slabă a datelor
- lipsa suportului pt. căutarea datelor
- acces liniar la un fișier text
- protecția generală la nivel de sistem de operare NU e dezavanta

16. Dacă avem schema:
Persons (pid: number, name: varchar, age: number)
unde pid = cheie primară
interogarea folosită pt. a face loc unei noi înregistrări care va avea pid = 1 : UPDATE Persons SET pid=pid+1 ORDER BY pid DESC

Explicație: pid-ul e unic
dacă avem deja în tabel 3 pers. cu pid-uri 1, 2, 3 și nu apelăm DESC, ia primul rând (cu pid = 1) și vrea să-l facă pid=2 (1+1), dar nu poate pt. că există deja o pers. cu pid = 2
dacă băgăm DESC, înainte de toate le ordonează și ar veni 3, 2, 1, apoi ia pid = 3 => devine 4, 2 devine 3, 1 devine 2, iar 1 nu mai e folosit ca pid fiind liber și poate fi folosit

17. Integritatea referențială: o cheie externă dintr-un tabel secundar trebuie să indice mereu către o cheie primară validă din tabelul principal

Într-o bază de date relațională trebuie să se verifice integritatea referențială când:
- când se modifică val. unei chei primare sau externe
- când se șterge o înreg. din tabelul principal
- când se adaugă o înreg. în tabelul secundar.
- NU e necesar când se șterge o înreg. din tabelul secundar

18. Operatorul JOIN din alg. relațională poate fi definit folosind Selecție și Produs Cartezian

Scanned with CamScanner

19. Servicii care sunt oferite de DBMS:
- Mecanisme de protecție și recuperare a datelor
- Algoritmi avansați de căutare și extragere
- Organizare fizică eficientă a datelor pe medii de stocare ext.

→ NU oferă mecanisme avansate de versionare a codului

20. Având schema:

Sailors (sid: number, nome: varchar, ranh: number, salary: number age: number)

Expresia de Update reversibilă e:

UPDATE Sailors SET salary = salary + 10/100 * salary (pentru că putem reveni la salariile inițiale matematic)

La celelalte e ireversibilă (se pierd complet datele):

UPDATE Sailors SET
- nome = "..." ... (se pierde numele)
- ranh = 1 (se pierde ranh-ul)
- ranh = ranh + 1 WHERE ranh < 9 (se suprapun ranh-urile când vrei să modifici)

→ partea 2_bd

21. Mecanisme de autentificare:

KERBEROS, RADIUS, PKI } suportate de Oracle

DIAMETER - NU e suportat

22. Criteriile multiple dintr-o listă de parametri sunt interpretate într-o manieră ierarhică în ORDER BY, dar nu și în GROUP BY

23. Lista de proiecție a unei interogări SELECT (adică ce e scris după SELECT) poate conține un atribut al tabelului și o funcție de agregare SQL doar dacă atributul e inclus în clauza GROUP BY.

24. O funcție de agregare (ex. SUM, AVG) nu poate fi folosită în mod direct în clauza WHERE.

25. Interogările nu sunt corelate pt. că în subinterogarea din clauza IN se folosesc tabelele s1, b1, r1, fără nicio referire la cele din afară (s, b, r)

26. SELECT ... WHERE ranh > ALL (SELECT ranh FROM Sailors) returnează doar ranh-urile care sunt mai mari decât toate ranh-urile returnate de subinterogare

(dacă se compară o coloană cu ea însăși cu >ALL mereu va fi gol)

Scanned with CamScanner

27. Dacă un tabel care conține un singur rând în care toate atributele, în afară de cheia primară (PK), conțin valori NULL, SELECT COUNT(*) FROM Table returnează 1 (pentru că COUNT * numără rândurile și avem un rând, dacă era COUNT de altceva și acel ceva zicea că e NULL, ar fi returnat NULL)

28. Pentru a fi compatibile la reuniune, două relații (tabele) trebuie să aibă același nr. de atribute (coloane), cu tipuri de date corespondente, în aceeași ordine.

29. FULL JOIN poate fi implementat folosind UNION (Reuniune)

30. Un index de tip arbore, cheia compusă $<x_1, x_2, x_3, x_4>$ NU poate fi folosit pt. a găsi potriviri în selecția $x_3 = 7$

! pentru că se poate folosi doar dacă elementele sunt în aceeași ordine-n caută ca și în cheie

ex: $x_1 = \ldots$ and $x_2 = \ldots$

$x_1 = \ldots$

31. WHERE age > (SELECT s.age ..) — compară cu mai multe

≥ - operator scalar, se poate compara cu o singură valoare
mergea dacă aveam age > ALL (...) sau age > ANY(...)

32. O subinterogare folosită ca operand al operatorului IN trebuie să returneze o mulțime de valori scalare.
(IN verifică dacă o valoare individuală se regăsește în interiorul unei liste de elemente)

33. Parametrul DISTINCT nu are niciun efect pentru fct. de agregare MAX.

34. Un index de tip hash, cheia $<x_1, x_2, x_3, x_4>$ poate fi folosit pt. a găsi potriviri în selecția $x_1 = \ldots$ and $x_2 = \ldots$ and $x_3 = \ldots$ and $x_4 = \ldots$

! la hash index - potrivire exactă (toate atributele)

! la index arbore - în aceeași ordine ca și-n cheie (nu neapărat toate)

35.
```
SELECT s.name FROM Sailors s
  WHERE NOT EXISTS (SELECT * FROM Sailors s1
                    WHERE s1.rank < s.rank)
```

- selectează marinarii pentru care nu există niciun alt marinar (s1) cu un rank mai mic decât al lor

Scanned with CamScanner

36. Costul unui plan de execuție a unei interogări nu depinde de numărul de atribute (coloane) ale relațiilor de intrare.

37. O subinterogare nu poate fi inclusă în ORDER BY.

Scanned with CamScanner